
Detecting and Splitting Quran Recitation in Noisy ASR Transcripts: A Competition Proposal

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Before an application can tell a learner what they recited wrongly, it has to work
2 out what they recited at all: which ayahs appear in a noisy ASR transcript, where
3 each one begins and ends, and whether the audio was Quran. We propose a
4 competition on that task and release the benchmark it needs: 258 human-reviewed
5 transcripts of production recordings, 1,267 gold ayah chunks over 604 ayahs, a
6 deterministic recording-level split whose oracle floor is exactly zero, a fixed sub-
7 second scorecard, a public 26-recording review sample, checksums, and three
8 evaluated baselines spanning an order of magnitude, one of them the hand-built
9 pipeline of a deployed application.

10 1 The task and why it is open

11 Hundreds of millions of Muslims memorize Quran, almost always with a listener who corrects them.
12 Applications that stand in for that listener transcribe the recitation with commercial ASR and must
13 then make sense of noisy text: the isti'adhah and the basmala precede the recitation without belonging
14 to any ayah, learners repeat words after catching a breath, restart, stutter and correct themselves, and
15 the ASR garbles what it cannot resolve. A learner may recite three ayahs, skip one and come back to
16 it, and nothing marks where an ayah ends.

17 Given a transcript, a system must return the set of ayah ids recited and an assignment of every
18 transcript word to one of them, or abstain when the audio was not Quran at all. Existing public Quran
19 speech resources target ASR of professional read recitation, where the text is known in advance;
20 to our knowledge none benchmarks recovering the reference from an unconstrained memorization
21 attempt. Yet every Quran-memorization application solves this, badly, in private.

22 2 The benchmark

23 **Source and consent.** The 258 recordings come from adult users of a Quran-memorization chat bot,
24 under terms of service that disclose review for service improvement. A commercial ASR model
25 produced the transcripts, and a human reviewed every row after LLM pre-labeling, assigning ayah
26 ids, splitting the transcript and recording a confidence. 254 rows are Quranic, covering 604 distinct
27 ayahs from 45 surahs, with 1–46 ayahs and 2–261 words per recording (median 40); the remaining
28 four are non-Quranic negatives on which a system must abstain. We release text only, replace learner
29 and recording identifiers with surrogate ids, and checked the negatives by hand. Since the recited text
30 comes from a closed public canon, the transcripts carry essentially no personal information. Licence
31 CC BY 4.0.

Split. A deterministic stratified 60/40 split by *recording*, never by chunk: 151/107 recordings and 711/556 chunks. Splitting by chunk would leak one reciter’s ASR quirks across the boundary. The negatives split 2/2 so abstention is testable held out, and the two rows whose gold assignment spans surahs are pinned to train, which makes the held-out oracle floor exactly 0: feeding the gold split back scores zero, so the dataset is internally consistent and the metric has a known bottom.

Sample and integrity. A 26-recording sample, ten percent of the set, is published for review. It carries both negatives, a non-contiguous recitation, a medium-confidence row, a long al-Baqarah range and a recording of over 120 words, so a reviewer sees every behaviour the metric scores. Every released file is pinned by SHA-256.

3 Metric and baselines

The scorecard is a fixed script; lower is better; each of the three components lies in $[0, 1]$ and is printed beside the scalar: `research_score = (1 - ayah-set match) + (1 - word-assignment accuracy) + (1 - abstention accuracy)`. Detection compares the *set* of ayah ids, so a repeated ayah is a split concern rather than a detection error. Split accuracy is the fraction of gold transcript words placed in the correct ayah, compared after a metric-only orthographic canonicalization that folds hamza-bearing alif forms, alif maqsura and hamza seats and strips diacritics, so spelling noise is not scored as a segmentation error. An empty system scores 2.0 and the gold labels 0.0, and scoring the full set takes under a second, which matters for the agent track below.

Baseline	full set	held-out	held-out components	
Empty system	2.000	2.000	det 1.00, split 1.00	abstain 0
Incumbent application pipeline (hand-built)	0.785	0.760	det 0.17, split 0.09	abstain 0.50
Best of six coding-agent runs ¹	0.008	0.079	det 0.06, split 0.02	abstain 0

Table 1: ~~Baselines on the released split. The incumbent is the segmentation stack of a deployed application, tuned over roughly two years.~~

The spread is the point. The incumbent is not a strawman: it is what the application ships, and it still misses 17% of ayah sets and abstains on only one of the two held-out negatives. Six autonomous runs of two coding agents in a Karpathy-style fixed-metric loop all matched or beat it held out, the best by an order of magnitude.¹ That study is also why this competition keeps a hidden final test: without a held-out split the loop drove the score ten times lower largely by memorizing evaluation rows, hardcoding up to 41 ayah ids per run including literal transcript-to-answer lookups, and merely disclosing that a held-out set existed eliminated the literal memorization. A public score on a public split is not evidence of a better algorithm.

The residual headroom is concentrated: cross-surah recitations, non-contiguous ayah sets, and abstention, where a single missed rejection costs half a point and rare-event robustness separates otherwise indistinguishable systems.

4 Competition plan, feasibility and ethics

The competition launches on the frozen benchmark with a public leaderboard on the released test split, and ranks on a hidden test set of recordings collected after the public release, labeled by the same protocol and published when the competition closes. A read-only production check has identified 63 further recordings, 57 textually distinct from the released export, as the start of that collection; reuse permission and de-identification are verified first. A submission is a single Python file, and scoring takes under a second, so an *agent track* for autonomous loops with per-experiment logs runs beside the human track under identical rules, with hardcoded-constant counts reported for every entry.

Feasibility. Harness, data, checksums and review sample are attached, and the baselines run offline without API keys or audio. **Ethics.** Text only, de-identified, adult users, disclosed review, CC BY 4.0; the task supports corrective feedback rather than surveillance, and abstention is scored precisely so a deployed system is rewarded for staying silent when it does not understand. **Participation.** Small data, sub-second evaluation, and a problem every Quran-learning application faces; Arabic NLP, speech and coding-agent groups are natural entrants. A companion submission covers the within-ayah mistake annotation built on the same recordings.

¹Autoresearch with coding agents: generalizers and metric-maximizers on Quran recitation data, AIST 2026.